Project Proposal

Chase Rush

A Top-Down Police Chase Survival Game

Programming 2 | Kasetsart University (CPE)

1. Project Overview

Chase Rush is a top-down 2D survival game developed using Python and Pygame. The player controls a car navigating an open environment while being chased by police vehicles. The player's score is determined by how long they survive — the longer they evade capture, the higher the score. The game increases in difficulty over time, with more police units deployed and obstacles becoming more frequent. The project includes both a playable game module and a statistics dashboard that collects and visualizes gameplay data.

2. Project Review

Reference Project: Smashy Road: Wanted 2 (Bearbit Studios, 2017)

Smashy Road: Wanted 2 is a top-down car chase game where players drive various vehicles while being pursued by police. The core loop involves surviving as long as possible while destroying police vehicles and collecting power-ups. The game features procedurally generated environments and a large roster of unlockable vehicles.

Key improvements in Chase Rush:

- Statistics & Analytics: Unlike Smashy Road, Chase Rush collects detailed gameplay statistics and provides a data visualization dashboard — making the game suitable for academic analysis.
- Designed for Desktop: Optimized for keyboard-based controls on PC using Pygame, unlike the mobile-first design of Smashy Road.
- Transparent Difficulty Scaling: Police spawn rates and speed increase at defined intervals that the player can observe, giving clearer feedback on progression.
- Data-Driven Design: Gameplay parameters (speeds, spawn rates) can be tuned based on collected statistical data for better game balance.

3. Programming Development

3.1 Game Concept

The player pilots a car on a scrolling top-down map. Police cars spawn at the edges of the map and chase the player using pathfinding logic. The game ends when a police car collides with the player. The score is calculated based on survival time in seconds. As time progresses, police become faster and more numerous. The player can use obstacles such as buildings and barriers to break line-of-sight with pursuing police.

Key Features:

- Survival-based scoring (score = time survived in seconds)
- Progressive difficulty — more police spawned over time
- Top-down 2D rendering with Pygame
- Real-time statistics collection during gameplay
- Post-game stats dashboard with visualizations

3.2 Object-Oriented Programming Implementation

The following classes will be implemented:

Class	Key Attributes / Methods	Role
Game	run(), update(), render(), handle_events()	Main game loop controller. Manages game state, clock, and coordinates all subsystems.
Player	x, y, speed, directionmove(), draw(), get_rect()	Represents the player's car. Handles input-based movement and collision detection boundary.
Police	x, y, speed, targetchase(), draw(), update()	Represents a pursuing police vehicle. Uses simple chase AI to follow the player's position.
Map	tiles, obstaclesgenerate(), draw(), get_obstacles()	Manages the game world. Renders the background tiles and maintains obstacle positions.
StatsTracker	session_data, recordslog_event(), save_csv(), get_summary()	Collects gameplay metrics each frame/event and saves them to a CSV file at game over.

Dashboard	data_frameload_data( ), plot_charts(), show()	Loads saved CSV data and renders statistical visualizations using matplotlib after the game ends.

3.3 Algorithms Involved

- Chase AI (Simple Pursuer): Each Police object calculates the angle toward the player's position each frame and moves in that direction. This simulates basic homing pursuit without full pathfinding.
- Collision Detection: Rectangle-based collision (pygame.Rect.colliderect) is used to detect player-police and player-obstacle interactions.
- Difficulty Scaling: A time-based event system spawns additional police units at set intervals (e.g., every 30 seconds) and incrementally increases their speed.
- Data Logging Pipeline: The StatsTracker class records values every frame or game event and writes structured data to a CSV file using Python's csv module.

4. Statistical Data (Prop Stats)

4.1 Data Features

The following 5 features will be tracked every frame (60fps) during each gameplay session:

Feature	Why it is good to have this data?	How will you obtain 100 values?	Which variable/class will collect this?	How will you display this?
Player Position (x, y)	Shows movement patterns and areas where players tend to survive or get caught	Logged every frame — 100+ records in under 2 seconds	StatsTracker (player.x, player.y)	Scatter plot (position heatmap)
Player Speed	Reveals whether players move fast or slow under pressure	Logged every frame	StatsTracker (player.speed)	Line chart, Box plot
Distance to	Measures how close the player	Logged every frame	StatsTracker (distance calculation)	Line chart, Histogram

Nearest Police	gets to police over time			
Number of Active Police	Shows how difficulty scales over time	Logged every frame	StatsTracker (len(police_list))	Line chart
Player Direction (angle)	Shows turning behavior and evasion strategy	Logged every frame	StatsTracker (player.direction)	Histogram

4.2 Data Recording Method

Gameplay data will be saved to a CSV file (gameplay_stats.csv) using Python's built-in csv module. The StatsTracker class will append one record per frame (60fps). All features including player position, speed, direction, distance to nearest police, and number of active police will be logged every frame throughout the entire gameplay session.

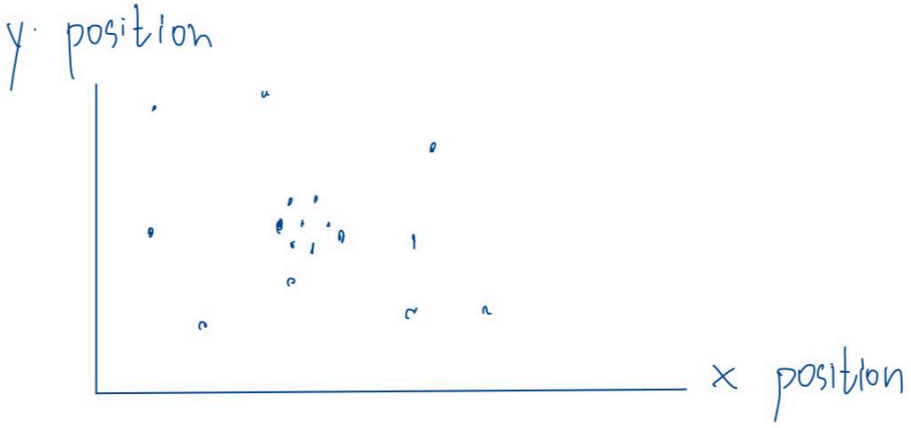
4.3 Data Analysis Report

The Dashboard class will load the CSV file using pandas and generate visualizations using matplotlib:

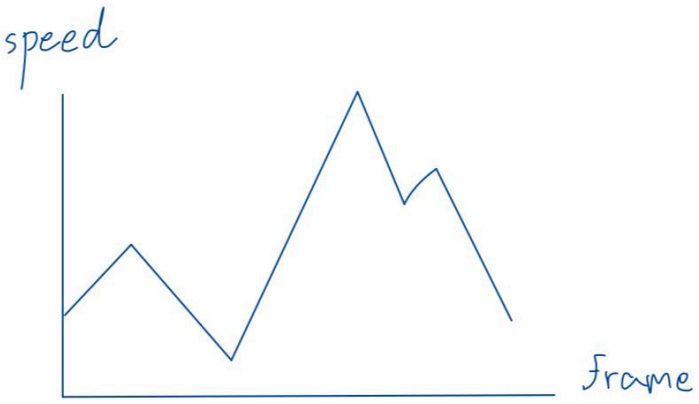
Graph	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	Player Speed	Show speed distribution	Histogram	Speed value	Frequency
Graph 2	Distance to Nearest Police	Show distance distribution	Box plot	-	Distance (px)
Graph 3	Player Position	Show distribution of player x-position across the map	Histogram	X position	Frequency
Graph 4	Player Speed	Show speed change over time	Line chart	Frame	Speed
Graph 5	Number of Active Police	Show difficulty scaling	Line chart	Frame	Police count

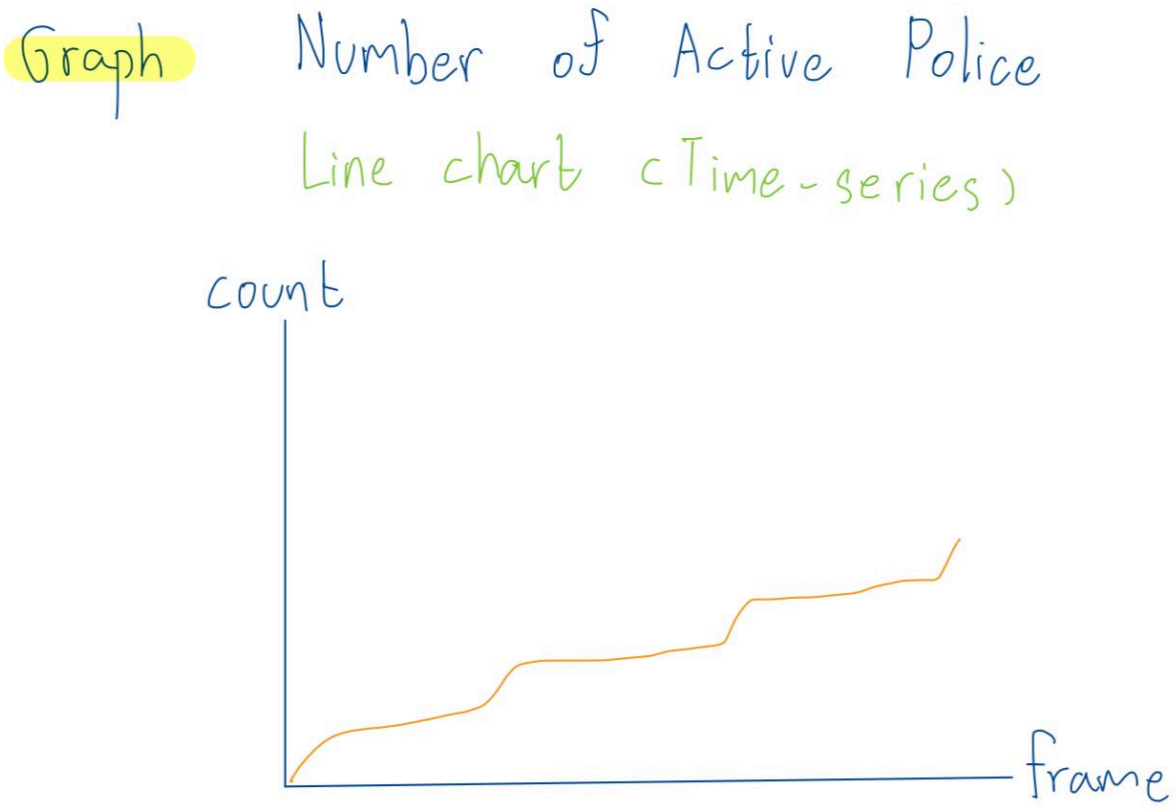
Feature	Mean	Median	Mode	Max	Min	Std Dev
Player Speed	-	-	-	-	-	-
Distance to Nearest Police	-	-	-	-	-	-
Number of Active Police	-	-	-	-	-	-
Player Direction (angle)	-	-	-	-	-	-
Player Position (x)	-	-	-	-	-	-

Graph Player position heatmap Scatter plot (Relation)



Graph Player speed overtime Line chart (Time series)





5. Project Planning Timeline

Week	Course Week	Task
1	8	Proposal submission / Project setup, GitHub repo initialization
2	9	Implement Player, Map, basic rendering with Pygame
3	10	Implement Police class with chase AI, collision detection
4	11	Implement StatsTracker, difficulty scaling system
5	12	Implement Dashboard with data visualization (matplotlib)
6	13	Testing, bug fixing, data collection (100+ records)

7	14	Submission week (Draft) — final report, UML diagrams, video presentation

6. Document Version

Version	1.0
Date	13 March 2026
Editor	Kawin Kaewparadai 6710545431

Date	Name	Feedback
16/03	Amornrit	The core concept sounds good. Your proposal is well-organized and formatted. However, you may need to add more features or details to give your project more depth. Explain more details about obstacle types. Moreover, as the police use basic homing pursuit, could they move through obstacles? You mentioned “collecting power-ups” and “a large roster of unlockable vehicles”, but I don’t see any details or classes and attributes related to them. In data features, **Survival Time** and **Police Count at Game Over** can be useful, but it would be a tedious job, as you have to play the game a hundred times to meet the minimum requirement (at least 100 records per feature). I recommend focusing on features that can hit 100 records within just a few plays to save time.
17/03	Peraya	I agree with P’Amornrit. Also this formatting is similar and I think I know where you are getting all this from. I hope this project shows what your work skill is. **Major Changes Needed** - **Section 3.2:** The current class structure is “flat” and lacks the complexity. You have single, isolated classes for Player and Police. This fails to demonstrate Inheritance or Polymorphism. - **Section 4.1** : As highlighted by P’Amornrit, your data collection strategy is logically flawed for a Year Project. You listed "Survival Time" and "Police Count at Game Over" as primary features. These are Session Level metrics (1 record = 1 game). To reach the 100-record requirement,

		you would have to play the game 100 times. This is unacceptable and inefficient. **Minor Changes Needed** - **Section 3.3 Algorithms**: You mentioned “Simple Homing” AI. In a map with buildings (obstacles), police will get stuck against walls, making the game unplayable or broken. - **Section 3.2 Game Features:** Your review mentions "collecting power-ups" and "unlockable vehicles," but your class list (3.2) does not include an Item class or a VehicleManager. Either remove these mentions from the Review or add the corresponding Classes and Attributes to your Implementation section to ensure the proposal is cohesive.
04/04	Peraya	- You have a Graph Plan and a Feature Table, but you are missing the **Table of Statistical Values** (showing Mean, Median, Max, Min, etc.). The rubric requires at least 1 table specifically for showing statistical values. - You also choose the wrong graph type for **Player Position.** You choose use 2 bar graph or 3d bar graph for distribution.